

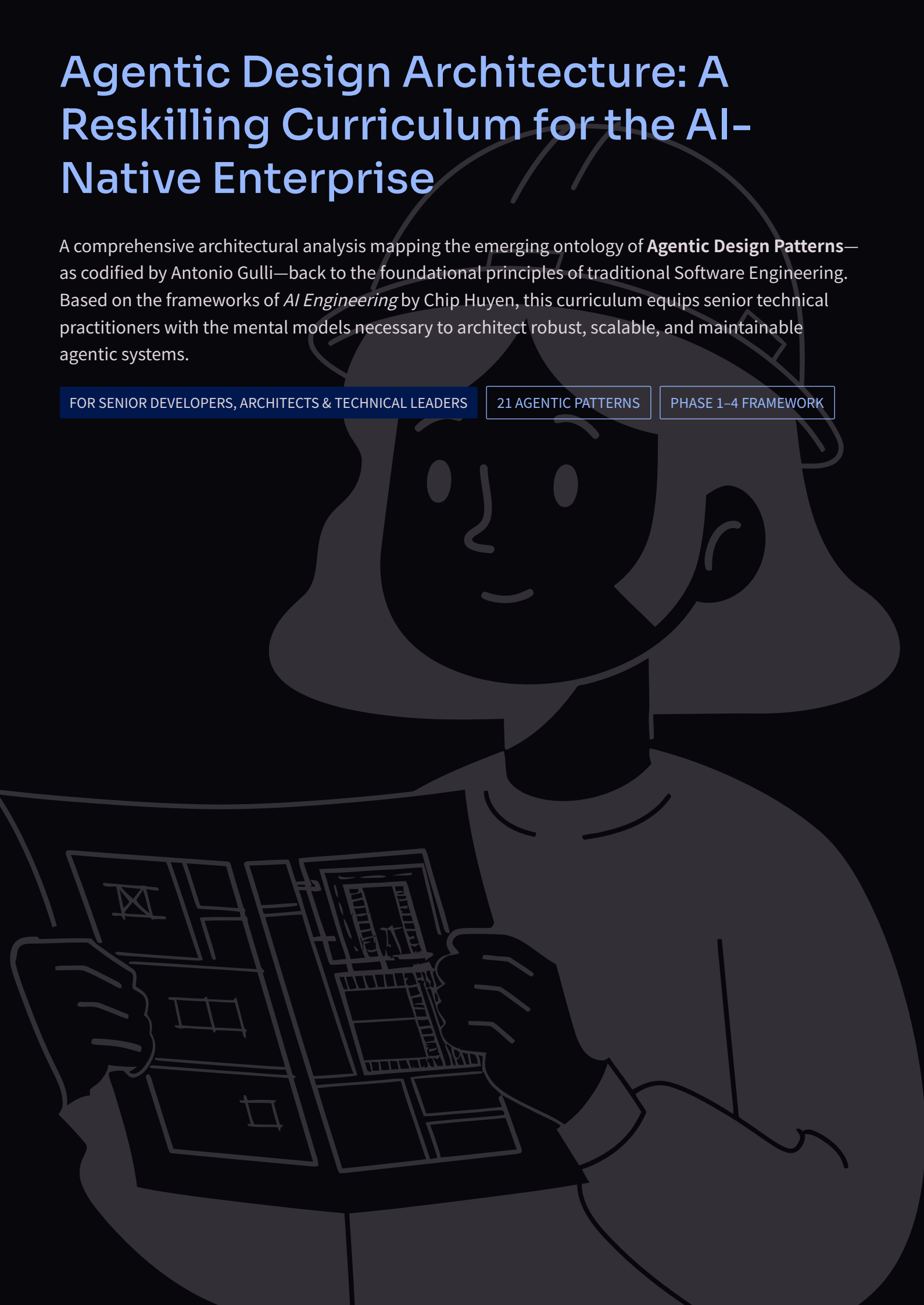
Agentic Design Architecture: A Reskilling Curriculum for the AI-Native Enterprise

A comprehensive architectural analysis mapping the emerging ontology of **Agentic Design Patterns**—as codified by Antonio Gulli—back to the foundational principles of traditional Software Engineering. Based on the frameworks of *AI Engineering* by Chip Huyen, this curriculum equips senior technical practitioners with the mental models necessary to architect robust, scalable, and maintainable agentic systems.

FOR SENIOR DEVELOPERS, ARCHITECTS & TECHNICAL LEADERS

21 AGENTIC PATTERNS

PHASE 1–4 FRAMEWORK



Executive Summary: The Architectural Shift from Deterministic to Probabilistic Systems

The discipline of Software Engineering (SWE) is currently navigating its most significant inflection point since the transition from monolithic to distributed architectures. For the past forty years, the fundamental contract of software design has been deterministic: systems were engineered to produce predictable outputs from known inputs through rigid, immutable logic paths. The definitions of correctness were binary—a unit test either passed or failed; a state machine transition was either valid or invalid; a variable held a specific value or it did not.

We are now entering the era of **Agentic AI**, where this deterministic paradigm is being augmented—and in specific domains, superseded—by probabilistic architectures. In this new world, software does not merely execute pre-written instructions; it perceives, reasons, plans, and acts. The core processing unit has shifted from the Arithmetic Logic Unit (ALU) to the Large Language Model (LLM), and the primary control structure has evolved from the `if-else` statement to the semantic reasoning loop.

The Old Contract

- Deterministic outputs from known inputs
- Binary correctness (pass/fail)
- Rigid, immutable logic paths
- ALU as core processing unit
- `if-else` as primary control structure

The New Contract

- Probabilistic architectures augmenting determinism
- Qualitative correctness (helpfulness, accuracy)
- Dynamic, semantic reasoning loops
- LLM as core processing unit
- Plan-and-Execute as primary control structure

This research report serves as a comprehensive "Reskilling Curriculum" for Senior Developers, Architects, and Technical Leaders. It is not a guide to "AI Hype," but a rigorous architectural analysis that maps the emerging ontology of **Agentic Design Patterns**—as codified by Antonio Gulli—back to the foundational principles of traditional Software Engineering. The objective is to equip the senior technical practitioner with the mental models necessary to architect robust, scalable, and maintainable agentic systems. We will deconstruct **21 specific agentic patterns**, mapping each to a familiar SWE concept—from Load Balancers to Circuit Breakers—providing a bridge between the known world of deterministic code and the emerging world of probabilistic reasoning.

Phase 1: Foundational Frameworks — The Physics of Agency

Before analyzing individual implementation patterns, we must first establish the "physics" of this new domain. The transition from a script to an agent represents a change in the fundamental nature of the software's relationship with its environment and its user.

1.1 The Definition of 'Agency' vs. 'Agent'

In traditional software, a program is a passive tool. It possesses no volition; it waits for an external stimulus (a user click, an API request, a cron job) to trigger a specific, bounded operation. The concept of "Agency" introduces a layer of active engagement with the problem space.



The Model (The Engine)

An LLM (e.g., GPT-4, Claude 3.5 Sonnet, Gemini) is a probabilistic inference engine. It is a "brain in a jar." It has immense potential energy—reasoning capabilities, world knowledge, language processing—but zero kinetic energy. It cannot check the time, read a file, or send an email unless explicitly wrapped in a runtime environment. It answers, but it does not do.



The Agent (The System)

An agent is the architectural wrapper around the model. It is a system designed to perceive its environment, reason about that environment to formulate a plan, and execute actions to achieve a high-level goal. **Agency** is defined as the functional capacity to autonomously navigate a multi-step decision tree involving Perception, Cognition, and Action.



The OODA Loop

Unlike a standard script, an agent maintains a dynamic "OODA Loop" (Observe, Orient, Decide, Act). It does not just follow a path; it chooses a path based on the current context.

❏ **The SWE Parallel:** LLM ☒ CPU (raw processing power). Agent ☒ Process/Thread (the OS construct that owns resources, interacts with I/O, and manages the lifecycle of execution).

1.2 The Maturity Model of Agentic Autonomy

The industry is coalescing around a maturity model that categorizes systems based on their level of autonomy and reasoning complexity. This model helps architects assess where a specific application sits on the spectrum from "fancy autocomplete" to "autonomous employee."



Level 0: Reactive (Zero-Shot)

The system responds to a single prompt immediately. No memory of past interactions, no external actions. Purely an information retrieval or generation engine. *Example: A standard ChatGPT query: "Write a poem about rust."*



Level 1: Tool-Use (Function Calling)

The system can decide to invoke a specific external function (API) to retrieve data or perform a calculation before answering. It acts as a router between natural language and deterministic code blocks. *Example: "What is the weather in Tokyo?" → System calls `get_weather('Tokyo')`*



Level 2: Planning & Reasoning (Chain of Thought)

The system decomposes a complex, ambiguous goal into a sequence of logical steps before execution. It maintains a short-term plan state and executes steps sequentially. *Example: "Plan a travel itinerary for Paris." → Search flights, Search hotels, Check restaurants.*



Level 3: Context-Aware (Memory & Persistence)

The system maintains long-term state across sessions. It "remembers" user preferences, past decisions, and environmental context (Episodic Memory). *Example: A coding assistant that remembers your preferred variable naming convention from last month.*



Level 4: Autonomous Multi-Agent Collaboration

A system of systems. Multiple specialized agents (Researcher, Reviewer, Coder) collaborate, hand off tasks, debate, and self-correct without human intervention. *Example: "Build a clone of Flappy Bird." → Architect plans → Coder writes → QA tests → Deployment pushes to cloud.*

1.3 The Decision Framework: Deterministic Script vs. Agentic Workflow

One of the most critical responsibilities of the modern Software Architect is creating the "Decision Boundary." Not every problem requires an agent. In fact, applying agentic reasoning to a deterministic problem is an anti-pattern that introduces unnecessary latency, cost, and unpredictability. Chip Huyen's *AI Engineering* advocates for a "start-simple" approach. We can formalize this into an architectural decision matrix:

Architectural Characteristic	Deterministic Script (Traditional SWE)	Agentic Workflow (AI Engineering)
Problem Topology	Well-defined, bounded, repetitive. Solution path known at compile time.	Ambiguous, open-ended, dynamic. Solution path discovered at runtime.
Input Data	Structured (JSON, SQL, Protobuf). Predictable schema.	Unstructured (Natural Language, Images, Video). High variance and noise.
Control Flow	Rigid branching (if/else, switch). Hard-coded logic paths.	Probabilistic reasoning (Plan-and-Execute). Logic flows determined by semantic evaluation.
Error Handling	Exception catching (try/catch). Fail-fast or fallback to default.	Self-correction / Reflection. The system attempts to "think" its way out of the error.
Success Metric	Binary (Pass/Fail). Verified by assertions.	Qualitative (Helpfulness/Accuracy/Relevance) . Verified by evaluation pipelines (LLM-as-a-Judge).
Maintenance	Code refactoring.	Prompt engineering and Context management.

The Architect's Rule of Thumb: If you can write a flowchart for the process that never changes, build a deterministic script. If the flowchart requires a human to make a judgment call at any box, build an agentic workflow to simulate that judgment.

Phase 2 & 3: Pattern 1 — Prompt Chaining

SWE PARALLEL: PIPE AND FILTER / CHAIN OF RESPONSIBILITY

Prompt Chaining is the foundational design pattern where a complex task is decomposed into a linear sequence of smaller, discrete LLM calls. The output of one step becomes the input (context) for the next step. By breaking a monolithic prompt into a chain, the system reduces the cognitive load on the model for any single inference, significantly improving accuracy, adherence to instructions, and reliability. It allows for intermediate "gates" where deterministic code can validate outputs before passing them to the next link in the chain.

Similarity to SWE

Both patterns rely on passing data sequentially through processing nodes. Each node acts independently, having a single responsibility (SRP). The system is composed of modular components that can be tested and optimized in isolation.

Key Divergence

In SWE, the transformation is deterministic (bytes in, bytes out) and the interface is rigid (data types). In Prompt Chaining, the transformation is probabilistic (text in, text out). The "interface" between nodes is natural language, which is "fuzzy" and unstructured. This necessitates a new type of "Type Safety"—often implemented via intermediate parsing or "Guardrail" agents.

Production Considerations

Latency

Latency is additive: **Total Latency = $\Sigma(\text{Step}_1 \dots \text{Step}_N)$** . Chains can become slow if they grow too long. Optimize prompt length and generation length at each step.

Reliability

Error propagation is a significant risk. If Step 1 hallucinates, Step 2 processes garbage ("Garbage In, Garbage Out"). Validation gates between steps are critical reliability engineering.

Cost

While multiple calls increase request count, breaking a task down can actually reduce total token usage compared to a massive, multi-turn conversation requiring re-reading a huge context window.

- ❏ **Key Reskilling Takeaway — Unlearn:** The instinct to write a "Mega-Function" or a "Mega-Prompt" that handles all edge cases. **Adapt:** Think in terms of "Semantic Micro-Transactions." Break reasoning down into atomic units of work. Architect your prompt chains like you architect a data pipeline: Step A → Validate → Step B → Transform → Step C.

Patterns 2 & 3 — Routing and Parallelization

Pattern 2: Routing

SWE Parallel: Load Balancer / API Gateway / Strategy Pattern. Routing involves dynamically directing a user request to the most appropriate specialized agent, model, or processing path based on the *semantic intent* and *complexity* of the query. Instead of a single general-purpose model handling everything, a "Router" (often a smaller, faster model) classifies the input and delegates execution to a downstream handler optimized for that specific task.

Similarity: Both mechanisms exist to optimize resource usage, enforce separation of concerns, and ensure that a request is handled by the component best suited for it.

Divergence: Traditional routing is rule-based and syntactic (Regex, URL paths, headers). Agentic routing is semantic; it requires an LLM call or a vector similarity search to classify intent. This introduces a probabilistic element at the very entry point of the system—the router itself can "misunderstand" the destination.

Production: Using a frontier model (e.g., GPT-4) for the routing step is often overkill and expensive. A best practice is to use a smaller, faster model (e.g., fine-tuned Llama 3, Gemini Flash) or even a simple BERT classifier. The router is a single point of failure—evaluate it using a **Confusion Matrix** to track misclassifications.

Reskilling Takeaway: The "Controller" in your MVC architecture is now an AI. You are architecting a system where control flow is determined by natural language classification, not hard-coded logic paths. You must verify the accuracy of this "Semantic Switch."

Pattern 3: Parallelization

SWE Parallel: Scatter-Gather Pattern / MapReduce / Async-Await Concurrency. Parallelization involves executing multiple independent agentic tasks simultaneously—such as voting on a decision, generating multiple creative drafts, or verifying facts against different sources—and then aggregating the results. This is also known as the "Sectioning" or "Voting" pattern.

Similarity: Both patterns aim to reduce wall-clock time by utilizing concurrent processing threads. They both follow a "Fork-Join" topology where a main thread spawns workers and waits for their completion. **Divergence:** In SWE, aggregation is usually deterministic (summing numbers, merging arrays). In Agentic systems, aggregation requires a "Reducer" agent—a specialized LLM call—to synthesize disparate natural language outputs.

Production: Parallelization offers a dramatic reduction in end-to-end latency for complex tasks. However, costs scale linearly with the number of parallel branches. Running 5 agents in parallel costs 5x more tokens. You must also handle "stragglers" (agents that time out): does the aggregator fail if one thread fails (fail-all), or proceed with partial information (best-effort)?

Reskilling Takeaway: Concurrency is now applied to "thoughts" and "perspectives." You are architecting a digital boardroom where multiple entities "think" at once, and you need a strategy (the Reducer) to merge their conclusions into a single coherent output.

Patterns 4 & 5 — Reflection and Tool Use

Pattern 4: Reflection (Self-Correction)

SWE Parallel: Unit Testing / Code Review / TDD Loop.

Reflection enables an agent to critique its own output (or the output of another agent) to identify errors, hallucinations, or areas for improvement, and then iteratively refine the result. It acts as an internal feedback loop, significantly increasing the quality and robustness of the final output.

Similarity: The "Red-Green-Refactor" loop in TDD is structurally analogous to the "Draft-Critique-Revise" loop in Reflection.

Divergence: Unit tests check against *known, deterministic* assertions (e.g., `assert x == 5`). Reflection checks against *qualitative, ambiguous* criteria (e.g., "Is this tone professional?") using an LLM as the judge. The "test" itself is probabilistic.

Production: Reflection is expensive in terms of time—it can double or triple latency. There is also a risk of "degeneracy," where the agent critiques itself into a worse state or gets stuck in a loop of minor, inconsequential edits. Use cheaper models for drafting and stronger models for critique.

Reskilling Takeaway: You are embedding the "Code Review" process directly into the runtime application. The system self-corrects before the human ever sees the output. You must design the "Critique Prompts" as carefully as you write unit tests.

Pattern 5: Tool Use (Function Calling)

SWE Parallel: Adapter Pattern / Proxy Pattern / Dependency Injection.

Tool Use extends the capabilities of an LLM by allowing it to interact with the external world (APIs, databases, calculators, software environments) to perform actions or retrieve data that is not present in its parametric memory. This transforms the LLM from a text generator into a system controller.

Similarity: Just as the Adapter pattern wraps an incompatible interface to make it usable by a client class, Agentic Tool Use wraps an external API into a schema (JSON) that the LLM can "understand" and "call." **Divergence:** In SWE, the developer writes the code to call the API explicitly. In Agentic Design, the developer defines the *interface* (the tool definition), and the Agent *decides* when and how to call it. This is **Inversion of Control** at the logic level.

Production: Read-only tools (GET requests) are generally safe. Write-enabled tools (POST/DELETE, "Delete User", "Transfer Funds") require strict Guardrails and Human-in-the-Loop validation. If the tool API changes, the agent breaks—robust error handling is needed.

Reskilling Takeaway — Unlearn: "I write the logic that calls the function." **Adapt:** "I write the function description so the Agent knows how to use it." You are writing API documentation for a machine reader. The quality of your tool description determines the reliability of the system.

Patterns 6 & 7 — Planning and Multi-Agent Collaboration



Pattern 6: Planning

SWE Parallel: Workflow Orchestration / Saga Pattern / Finite State Machine (FSM). Planning is the capability of an agent to break down a high-level, ambiguous user goal into a sequence of executable steps (a plan), and then execute them, maintaining the state of progress.

Planning — Key Details

Similarity: Both involve managing a sequence of dependent tasks to achieve a complex goal. Both must manage state (which step are we on?) and dependencies (Step B requires Step A). **Divergence:** Standard workflows (e.g., Airflow DAGs, AWS Step Functions) are **static** and defined at compile-time. Agentic plans are **dynamic** and generated at run-time by the model. The agent builds its own DAG based on the specific request, allowing it to handle novel situations the programmer did not foresee.

Production: Agents often get stuck in loops or generate invalid plans (e.g., skipping a prerequisite). Evaluating planners is difficult—you must verify not just the final output, but the validity of the *generated plan logic*. Planning also takes time upfront ("Time to First Token" is high).

Reskilling Takeaway: You are moving from "Imperative Programming" (telling the machine *how* to do it) to "Declarative Goal Setting" (telling the machine *what* to achieve). You build the planner, not the plan.



Pattern 7: Multi-Agent Collaboration

SWE Parallel: Microservices Architecture / Actor Model / SOA. This pattern involves structuring a system as a team of specialized agents (Researcher, Writer, Reviewer, Coder) that collaborate, hand off tasks, and communicate to solve complex problems.

Multi-Agent — Key Details

Similarity: Both involve the decomposition of a monolithic problem into specialized, decoupled components that communicate over a network/protocol. Both rely on the principle of **Separation of Concerns**. **Divergence:** Microservices communicate via rigid, defined APIs (Syntactic Contracts). Agents communicate via natural language conversations (Semantic Contracts). The "handshake" between agents involves negotiation and context sharing, not just data transfer.

Production: High latency due to multiple round-trips and "conversations" between agents. Tracing a request through a conversation between 5 agents is exponentially more difficult than tracing a monolith—requires specialized tracing tools (like LangSmith or Arize) to visualize the "conversation graph." Multi-agent chat loops can also burn tokens rapidly.

Reskilling Takeaway: Architecture involves designing the "org chart" of your digital workforce. Who talks to whom? What is the hierarchy? Is it a flat mesh or a hierarchical team?

Patterns 8 & 9 — Memory Management and Learning & Adaptation



Pattern 8: Memory Management

SWE Parallel: Database / Caching / Session State Management.

Mechanisms for agents to store, index, and retrieve information over time, spanning short-term (conversation context) and long-term (knowledge base/episodic) history. This gives the agent a sense of continuity and identity.

Similarity: Persistence of state is crucial for both. Short-term memory $\approx$ RAM/Redis (Session Store); Long-term memory $\approx$ Disk/SQL/NoSQL (Persistent Store).

Divergence: Agentic memory is often **Vector-based** (semantic similarity) rather than Key-Value or Relational. Retrieval is probabilistic (getting "relevant" memories based on embedding distance) rather than exact (getting "row ID 123").

Production: Storing everything in context windows is expensive. Efficient RAG and summarization strategies are needed to manage the "Context Window Economy." Long-term memory also risks storing PII indefinitely—implementation of "Forgetting" mechanisms and strict data governance is required.

Reskilling Takeaway: Database schema design is replaced by "**Information Retrieval Strategy**." You don't just query data; you curate context. You must decide what the agent *needs* to know to be effective.



Pattern 9: Learning and Adaptation

SWE Parallel: CI/CD Pipelines / A/B Testing / Online Learning Systems.

The ability of an agent to improve its performance over time based on feedback, user interactions, or new data, without full model retraining. This includes techniques like "In-Context Learning" (updating few-shot examples) or updating a knowledge base.

Similarity: Both represent the continuous improvement lifecycle of the system.

Divergence: In SWE, improvement requires a code commit, build, and deploy cycle. In Agentic systems, improvement can happen dynamically at runtime. The agent can "learn" a new rule by adding it to its system prompt or memory store, instantly changing behavior without a deployment.

Production: "Drift" is a major risk. An agent adapting to bad feedback (poisoning) can degrade quickly. Guardrails are needed to prevent the agent from learning incorrect behaviors. How do you version control "learned behaviors"?

Reskilling Takeaway: Software is no longer a static artifact; it is a living system. Observability must track "Behavioral Drift" in real-time.

Pattern 10 — State Management (Model Context Protocol – MCP)

SWE PARALLEL: IDL / REST API STANDARDS / USB-C STANDARD

A standardized protocol for connecting AI models to data sources and tools, ensuring consistent context and state exchange between systems. MCP acts as a universal translator between the LLM and external systems.

The Mapping

Similarity: Both provide a standardized contract for interoperability. Just as USB-C allows any device to connect to any charger, MCP allows any AI agent to connect to any data repository (GitHub, Slack, Drive) without custom glue code.

Divergence: MCP focuses specifically on exposing *context* (prompts, resources, tools) to LLMs, optimizing for the way models "read" interfaces (token-efficient schemas) rather than how compilers read code.

Before vs. After

Before (Custom Glue Code): Custom integration for every tool — specific GitHub logic, specific Slack logic, each requiring bespoke engineering effort.

After (MCP Standard): MCP Client connects to generic MCP Servers. The client is agnostic to what the server actually is. Tools are auto-discovered and injected into the agent context via `client.list_tools()` and `agent.bind_tools(tools)`.

Scalability

Drastically reduces the engineering effort to integrate new tools. Write the MCP server once, use it with any agent.

Security

Standardized protocols allow for centralized security policies (Pattern 18).

- 📌 **Key Reskilling Takeaway:** Stop building custom connectors. Adopt standards (like MCP) that treat AI tools as "plug-and-play" peripherals.

Patterns 11 & 12 — Goal Setting & Monitoring and Exception Handling & Recovery

Pattern 11: Goal Setting & Monitoring

SWE Parallel: Health Checks / Watchdogs / APM (Application Performance Monitoring).

Mechanisms for agents to define high-level objectives, break them into trackable sub-goals, and continuously monitor progress towards completion. The agent checks: "Am I closer to the goal than I was 5 minutes ago?"

Similarity: Monitoring system health and progress toward a defined state. **Divergence:** Traditional monitoring checks system metrics (CPU usage, latency, error rate). Agentic monitoring checks **semantic progress** (e.g., "Have we gathered enough research data yet?", "Is the draft complete?"). It requires "Metacognition"—thinking about thinking.

Production: Essential to prevent agents from getting stuck in infinite loops ("doom loops") or declaring premature success. Monitoring also consumes tokens.

Reskilling Takeaway: You need to build "Manager Agents" whose only job is to supervise "Worker Agents." You are automating middle management.

Pattern 12: Exception Handling & Recovery

SWE Parallel: Try-Catch Blocks / Circuit Breaker Pattern / Retry-Exponential Backoff.

Strategies for agents to detect failures (e.g., API errors, hallucinations, logic loops) and self-correct or retry using alternative strategies. This allows the system to be resilient in the face of uncertainty.

Similarity: Preventing system crash upon encountering an error. **Divergence:** Agentic recovery is **semantic**. If a tool fails (e.g., "API Error 500"), the agent can reason about why and try a different tool (e.g., "Google Search failed, I will try Wikipedia") or rephrase the query. It is dynamic problem solving, not just retrying the same operation.

Production: Without this, agents are brittle toys. Real-world APIs fail; agents must navigate these failures autonomously. Ensure error recovery doesn't spiral into an infinite retry loop of different failed strategies.

Reskilling Takeaway: Error handling is now a creative problem-solving process. You don't just catch the error; you empower the agent to find a workaround.

Pattern 13 — Human-in-the-Loop (HITL)

SWE PARALLEL: MANUAL APPROVAL GATES (CI/CD) / MAKER-CHECKER PATTERN / TWO-FACTOR AUTHENTICATION

Integrating human oversight at critical decision points to validate agent actions, provide feedback, or disambiguate requests. The agent pauses execution and requests input from a human "Tool."

The Mapping

Similarity: Pausing a workflow for authorization or verification.

Divergence: In Agentic systems, the human is treated as a "Tool" or an **Oracle**. The agent generates a prompt for the human ("I plan to send this email. Is it okay?"), and the human's response is injected back into the agent's context as a tool output. The interaction is conversational.

Production Considerations

- **Safety:** The ultimate guardrail for high-stakes actions (writes, payments, public posts).
- **Latency:** Introduces massive latency (human speed vs. machine speed). Use asynchronous notifications (webhooks/Slack) rather than blocking the thread.

The agent proactively asks: *"I am about to delete a production database. Proceed?"* — The human is a component in the system loop. Design interfaces for the AI to "talk" to the human, not just the human to talk to the AI.

- ❏ **Key Reskilling Takeaway:** Design interfaces for the AI to "talk" to the human, not just the human to talk to the AI. The human is a component in the system loop.

Patterns 14 & 15 — Knowledge Retrieval (RAG) and Inter-Agent Communication (A2A)

Pattern 14: Knowledge Retrieval (RAG)

SWE Parallel: Database Querying / Search Index Integration (Elasticsearch).

Retrieval-Augmented Generation (RAG) equips agents with the ability to query external knowledge bases (Vector DBs, Search Indices) to ground their answers in fact and access proprietary data.

Similarity: Fetching data from storage to populate a view or answer a request.

Divergence: RAG involves **semantic search** (vector embeddings) rather than keyword match. It creates a **prompt context**, not just a data object. The "Schema" is the semantic meaning of the data chunks.

Before (SQL Query): `SELECT answer FROM faq WHERE topic = 'pricing'`

After (Agentic RAG): `Semantic retrieval via retriever.get_relevant_documents("What is the pricing model?") → Augment Context → Generate Answer.`

Production: You must evaluate the **Retrieval** (Did we get the right doc?) and the **Generation** (Did the LLM use the doc correctly?). This is known as "RAG Triad" evaluation. Vector search + LLM generation is also slower than a DB lookup.

Reskilling Takeaway: "Data Access Layer" now implies Vector DBs and Embedding Models. You are optimizing for "Context Relevance," not just query performance.

Pattern 15: Inter-Agent Communication (A2A)

SWE Parallel: Message Queues (RabbitMQ/Kafka) / API Contracts (REST/gRPC).

Protocols and formats for autonomous agents to exchange messages, tasks, and state information to collaborate on a shared goal.

Similarity: Decoupled components exchanging information to coordinate work.

Divergence: Agents talk in natural language or structured JSON schemas representing "Intent" and "Content." The communication is often a negotiation ("Can you help with X?" "No, I am busy, ask Agent Y") rather than a command.

Before (Structured API Payload): `POST /order {"id": 123, "action": "ship"}`

After (Agent Protocol): `{"sender": "SalesAgent", "recipient": "FulfillmentAgent", "content": "Client X wants the premium package. Can you verify stock?", "intent": "QUERY", "context_id": "session_789"}`

Production: Protocols like A2A (Agent-to-Agent) or Google's Agent protocols are emerging to standardize this handshake to prevent "Tower of Babel" scenarios.

Reskilling Takeaway: Define communication *protocols* and *personas*, not just data schemas. You are defining how digital employees talk to each other.

Patterns 16 & 17 — Resource-Aware Optimization and Reasoning Techniques

Pattern 16: Resource-Aware Optimization

SWE Parallel: Garbage Collection / Connection Pooling / Auto-scaling / Load Shedding.

Agents aware of their token consumption, API costs, and computational limits, optimizing their strategies accordingly (e.g., using a cheaper model for simple summarization vs. a frontier model for reasoning).

Similarity: Managing finite system resources (memory, CPU, budget) to prevent outages or overruns. **Divergence:** Optimization is **decision-based** (dynamic choice of model/path) rather than infrastructure-based (adding servers). The agent chooses to be frugal.

After (Resource Aware Routing):

```
if task.complexity == "LOW":
    model = "gpt-3.5-turbo" # Cheap, Fast
else:
    model = "gpt-4" # Expensive, Smart
```

Production: Critical for business viability. "Token economics" is a new architectural constraint. Switching models can also reduce latency for user-facing interactions.

Reskilling Takeaway: Treat "Intelligence" as a metered utility with variable cost tiers. Architect systems that use the "Least Capable Model Necessary" for the task.

Pattern 17: Reasoning Techniques

SWE Parallel: Algorithms / Design Patterns (e.g., Breadth-First Search, Recursion).

Advanced cognitive architectures like **Chain-of-Thought (CoT)**, **Tree-of-Thought (ToT)**, and **ReAct** (Reason+Act) that structure the model's internal processing to solve complex problems.

Similarity: The "method" by which the problem is solved. ToT is essentially a search algorithm (BFS/DFS) applied to the space of "thoughts."

Divergence: Reasoning is **prompted**, not coded. You don't write the for loop; you tell the model *how to think* about the loop. You are programming the *cognitive process*, not the instruction set.

After (Cognitive Prompting):

```
prompt = """
Solve using Tree of Thought:
1. Generate 3 possible next steps.
2. Evaluate each for feasibility.
3. Discard impossible paths.
4. Expand the best path.
Let's think step by step.
"""
```

Production: More reasoning steps = higher latency. CoT increases token count significantly. Use only when the task complexity demands it.

Reskilling Takeaway: Programming is now "Prompt Engineering" at the architectural level. You are defining the algorithms of thought.

Pattern 18 — Guardrails & Safety

SWE PARALLEL: INPUT VALIDATION / FIREWALLS / SECURITY POLICIES (IAM) / MIDDLEWARE

Architectural safeguards (input/output filters) to prevent agents from executing harmful actions, leaking PII, or deviating from policy. It ensures the agent stays "on rails."

The Mapping

Similarity: Preventing bad data or malicious actions from compromising the system.

Divergence: Guardrails must filter **semantic** risks (e.g., "Don't give financial advice," "Don't be rude") rather than just syntactic ones (e.g., "Drop SQL injection," "Validate Email format"). This often requires a separate, smaller LLM to act as the "Censor."

Before vs. After

Before (Input Sanitization):

```
if not valid_email(input):  
    throw Error()
```

After (Semantic Guardrail):

```
@OutputGuardrail  
def response = agent.generate()  
    safety_check =  
    guardrail_model.check(response)  
  
if safety_check.contains_pii or  
safety_check.is_toxic:  
    return "[Filtered]"  
else:  
    return response
```

Security

This is the "Firewall" of the AI age. Mandatory for enterprise compliance.

Latency

Adds latency to every request. Must be balanced against safety requirements.

- ❑ **Key Reskilling Takeaway:** Security is now probabilistic. You need "AI Firewalls" (Guardrail models) that can read and understand intent.

Patterns 19, 20 & 21 — Evaluation & Monitoring, Prioritization, and Exploration & Discovery

Pattern 19: Evaluation & Monitoring

SWE Parallel: Integration Testing / Observability / APM (Datadog/New Relic).

Frameworks for measuring agent performance (accuracy, faithfulness, tool usage) and monitoring behavior in production (traces, logs). **Divergence:** "Correctness" is subjective. Traditional metrics (Latency, Error Rate) are insufficient. You need "LLM-as-a-Judge" metrics to score "Correctness," "Hallucination Rate," and "Tone."

Before: `assert function(2, 2) == 4`

After: `score = evaluator.llm_grade(input, output, ground_truth) → Returns a score (e.g. 0.85) and reasoning → assert score > 0.9`

Production: Continuous evaluation in production is required to detect "drift" (model behavior changing over time due to updates or data changes).

Reskilling Takeaway: Testing is no longer binary. It is statistical. You are managing "Quality Assurance" via AI judges.

Pattern 20: Prioritization

SWE Parallel: Priority Queue / Job Scheduling / OS Process Scheduling.

The ability of an agent to rank disparate tasks or goals based on urgency, importance, and constraints, effectively managing its own backlog. **Divergence:** Priorities are determined by **semantic understanding** of the task content (e.g., "This email looks angry, prioritize it"), not just a numerical priority flag set by a user.

Before: `queue.push(task, priority=1)`

After: `priority_score = agent.evaluate_urgency(task_description) → Logic: "This is a CEO request, priority = Critical" → queue.push(task, priority=score)`

Production: Critical for agents managing their own time/resources in autonomous loops.

Reskilling Takeaway: The Scheduler is now intelligent. It understands the *content* of the work, not just the metadata.

Pattern 21: Exploration & Discovery

SWE Parallel: Chaos Engineering / Fuzz Testing / Web Crawling.

Agents proactively seeking new information, testing hypotheses, or exploring an environment to expand their knowledge or capabilities, rather than just reacting to prompts. **Divergence:** Exploration is **goal-directed** and **semantic**. The agent explores "concepts" or "solutions," not just code paths. It formulates a hypothesis, tests it, and learns.

After (Agentic Discovery):

Hypothesis Testing Loop — `while not hypothesis_proven(): experiment = scientist_agent.design_experiment(); result = lab_tool.run(experiment); scientist_agent.learn(result); scientist_agent.refine_hypothesis()`

Production: Unbounded exploration can be expensive and dangerous. Strict boundaries (sandboxing) are required.

Reskilling Takeaway: Systems can now self-evolve their understanding. You are the architect of the "Discovery Loop."

The 21 Patterns at a Glance: SWE Mapping Reference

#	Agentic Pattern	SWE Parallel	Core Divergence
1	Prompt Chaining	Pipe and Filter / Chain of Responsibility	Interface is natural language ("fuzzy"), not typed data
2	Routing	Load Balancer / API Gateway / Strategy Pattern	Routing is semantic (intent), not syntactic (URL/header)
3	Parallelization	Scatter-Gather / MapReduce / Async-Await	Aggregation requires an intelligent "Reducer" agent
4	Reflection	Unit Testing / Code Review / TDD Loop	The "test" itself is probabilistic, not deterministic
5	Tool Use	Adapter / Proxy / Dependency Injection	Inversion of Control — agent decides when to call
6	Planning	Workflow Orchestration / Saga / FSM	Plans are dynamic (runtime), not static (compile-time)
7	Multi-Agent Collaboration	Microservices / Actor Model / SOA	Communication is semantic (natural language), not syntactic (API)
8	Memory Management	Database / Caching / Session State	Retrieval is probabilistic (vector similarity), not exact
9	Learning & Adaptation	CI/CD / A/B Testing / Online Learning	Improvement happens at runtime, not via deployment
10	State Management (MCP)	IDL / REST Standards / USB-C	Optimized for token-efficient LLM "reading" of interfaces
11	Goal Setting & Monitoring	Health Checks / Watchdogs / APM	Monitors semantic progress, not just system metrics
12	Exception Handling & Recovery	Try-Catch / Circuit Breaker / Retry Backoff	Recovery is creative problem-solving, not retry-same-logic
13	Human-in-the-Loop (HITL)	Manual Approval Gates / Maker-Checker / 2FA	Human is a "Tool" / Oracle in the agent's context
14	Knowledge Retrieval (RAG)	Database Querying / Elasticsearch	Semantic search (embeddings), not keyword/SQL match
15	Inter-Agent Communication (A2A)	Message Queues / API Contracts	Communication is negotiation, not command
16	Resource-Aware Optimization	GC / Connection Pooling / Auto-scaling	Optimization is decision-based (model choice), not infra-based
17	Reasoning Techniques	Algorithms / Design Patterns (BFS, Recursion)	Reasoning is prompted, not coded
18	Guardrails & Safety	Input Validation / Firewalls / IAM / Middleware	Filters semantic risks, not just syntactic ones
19	Evaluation & Monitoring	Integration Testing / Observability / APM	Correctness is statistical (LLM-as-a-Judge), not binary
20	Prioritization	Priority Queue / Job Scheduling / OS Scheduling	Priority determined by semantic content, not metadata flag
21	Exploration & Discovery	Chaos Engineering / Fuzz Testing / Web Crawling	Exploration is goal-directed and semantic, not random

Phase 4: Synthesis — The Programmable Enterprise

The convergence of these 21 patterns signals a fundamental transformation in the role of the Senior Developer. We are moving away from defining *exactly how* a task is done (deterministic code) to defining *what success looks like* and providing the tools to achieve it (probabilistic agency).

This curriculum demonstrates that while the *paradigm* is new, the *architecture* is rooted in the best practices of the last decade. Load Balancers become Semantic Routers; Unit Tests become Evaluation Agents; Microservices become Multi-Agent Swarms. The "Interconnection Logic" is the key to mastering this transition.



Load Balancers

Become **Semantic Routers** that classify intent and direct traffic to specialized agents



Unit Tests

Become **Evaluation Agents** that score outputs using LLM-as-a-Judge pipelines



Microservices

Become **Multi-Agent Swarms** that collaborate via natural language semantic contracts

The Senior Developer of the future is an **Architect of Agency**. They will design the guardrails, the memories, the communication protocols, and the evaluation metrics that allow autonomous digital employees to operate safely and effectively within the enterprise. Welcome to the era of Agentic Engineering.

Key Reskilling Takeaways: The Architect of Agency

A consolidated reference of the core mental model shifts required for every senior practitioner navigating the transition to Agentic Engineering.

Think in Semantic Micro-Transactions

Unlearn the "Mega-Prompt." Break reasoning into atomic units of work. Architect prompt chains like data pipelines: Step A → Validate → Step B → Transform → Step C.

Your Controller is Now an AI

Control flow is determined by natural language classification, not hard-coded logic paths. You must verify the accuracy of this "Semantic Switch" using Confusion Matrices.

Embed Code Review into Runtime

Design "Critique Prompts" as carefully as you write unit tests. The system self-corrects before the human ever sees the output.

Write for the Machine Reader

Unlearn "I write the logic that calls the function." Adapt: "I write the function description so the Agent knows how to use it." Tool description quality determines system reliability.

Build the Planner, Not the Plan

Move from Imperative Programming (telling the machine *how*) to Declarative Goal Setting (telling the machine *what* to achieve).

Design the Org Chart of Your Digital Workforce

Architecture involves defining who talks to whom, the hierarchy, and whether it is a flat mesh or a hierarchical team. Define communication protocols and personas, not just data schemas.

Treat Intelligence as a Metered Utility

Token economics is a new architectural constraint. Architect systems that use the "Least Capable Model Necessary" for the task. Stop building custom connectors—adopt standards like MCP.

Testing is Now Statistical

Software is no longer a static artifact; it is a living system. Observability must track "Behavioral Drift" in real-time. You are managing Quality Assurance via AI judges. Security is now probabilistic—you need "AI Firewalls."

📖 **Sources:** Antonio Gulli, *Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems*; Chip Huyen, *AI Engineering*; Google Cloud Blog; Anthropic Research; AWS Prescriptive Guidance; Model Context Protocol (modelcontextprotocol.io); and additional references as cited throughout this curriculum.